Final Python Project - A Classification Project

This project counts as your final exam in Python.

In this project we want to predict the quality of a white wine based NOT on its region, grape or production year but rather on some chemical characteristics. The sweetness of a wine comes from sugar whereas the tartness comes from acidic components. White wines have much lower levels of tannins than red wine so this is not included as a feature here.

Suppose you have a good wine palette and only want to select wines which have a quality rating of 7 or higher on a scale of 1 to 10. So we want to turn this into a binary classification problem where we predict whether the wine is acceptable (1=True) or not acceptable (0=False) to purchase. The given data set provides several chemical features (see description below) and a quality rating between 1 and 10.

Once we have cleaned the data, separated it into training and test sets and then scaled it, we compare the 3 classification algorithms Logistic Regression, ANN and kNN for predicting whether a wine is acceptable or not based upon our criteria of a rating of 7 or higher.

Before fitting the data we need to put it in a data frame with the desired target value (0 or 1) and then we need to clean the data. To do this we complete the following steps.

1. We first determine if there are any data instances which have a feature value of NaN. If there are, delete these instances unless there are too many. If the majority of NaN's occur within a single feature, drop that feature.
2. Data instances which are *outliers* may skew the outcome of our algorithms so we want to delete such data, if possible. By an outlier we mean a data instance which has a feature value that is far from the mean. For example, for the residual sugar feature the mean is around 6.4 with a standard deviation of around 5. However, the maximum value occurring for residual is almost 66 so we may consider this data instance an outlier; note that its distance from the mean ($|6.4-66|=61.6$) is around 12 times its standard deviation so clearly something is wrong. The criteria we use to identify data entries as outliers is somewhat arbitrary but typically one requires the difference of the feature value from the feature mean to be less than some multiple of the standard deviation. In our case we will assume that points which are more than 5 standard deviations from the mean are outliers. So our criteria for determining if a data instance with feature value x is an outlier is $|x - \text{mean}| > 5$ times the standard deviation where mean and standard deviation are for the feature.

However, we do not want to delete too much data. If a feature has a large number of outliers then it is probably better to just not use that feature. In our data set we won't have to delete too many data instances so don't worry about deleting a feature. It is important to realize that in some applications, such as climate, outliers may be important and should not be deleted.

Dataset - White wine quality

This is a dataset which is used to predict the quality of a white wine based upon several features. The data is in the file 'white_wine_quality.csv'. However the data is NOT separated by a comma but rather by a semi-colon. This can be handled by the usual pandas command for reading a csv file except that you need to add the argument **sep=';'**

The features are:

1. fixed acidity (tartaric acid - g / dm^3): most acids involved with wine are fixed or nonvolatile (do not evaporate readily)
2. volatile acidity (acetic acid - g / dm^3): the amount of acetic acid in wine, which at too high of levels can lead to an unpleasant, vinegar taste
3. citric acid (g / dm^3): found in small quantities, citric acid can add ‘freshness’ and flavor to wines
4. residual sugar (g / dm^3): the amount of sugar remaining after fermentation stops, it’s rare to find wines with less than 1 gram/liter and wines with greater than 45 grams/liter are considered sweet
5. chlorides (sodium chloride - g / dm^3): the amount of salt in the wine
6. free sulfur dioxide (mg / dm^3): prevents microbial growth and the oxidation of wine
7. total sulfur dioxide (mg / dm^3): in low concentrations, SO2 is mostly undetectable in wine, but at free SO2 concentrations over 50 ppm, SO2 becomes evident in the nose and taste of wine
8. density (g / cm^3): the density of wine is close to that of water depending on the percent alcohol and sugar content
9. pH: describes how acidic or basic a wine is on a scale from 0 (very acidic) to 14 (very basic); most wines are between 3-4 on the pH scale
10. sulphates (potassium sulphate - g / dm3): a wine additive which can contribute to sulfur dioxide gas (S02) levels, wich acts as an antimicrobial and antioxidant
11. alcohol (% by volume): the percent alcohol content of the wine

The target/outcome in the data file is a number between 0 and 10 indicating the perceived **quality** of the wine. This data set may not contain wines from each quality rating.

What to do --

Step 0 Import libraries

Step 1 Load data and create a dataframe. Add a column to the data frame which gives the target value of 1 if the quality rating is 7 or higher and 0 otherwise; this answers the question "Is the wine acceptable?" based on our given criteria. Print out enough lines in your data frame to show that this column is correct. Hint: To create the extra column you can use the **.map()** method as we did many times with the iris data set or **.apply()** as we did in the diabetes dataset in Week 13

Step 2 Determine how many data instances are in the file; add a formatted print statement giving this value. Check to make sure there are no values containing NaN; if there are, delete those data entries.

Step 3 Decide how many different quality classes (based on the quality rating of 1 to 10) this data is divided into and how many wines are in each class. What quality rating does the majority of the wines in the dataset have? Give a frequency plot (using, for example, Seaborn's **countplot**) for each quality class and give the actual numbers in each quality rating. (The **groupby** method for a dataframe may be useful.)

Step 4 We now want to delete each data entry which has an outlier for any feature based upon the criteria we gave (i.e., the difference of a feature value from the feature mean is > five times the standard deviation). For each feature starting with 'fixed acidity' and going in feature order, loop through the data entries and see if the value is an outlier; if it is, delete that data entry. For each feature print out how many data instances you deleted. Remember that NumPy has built-in functions to determine the mean and standard deviation (**np.std(f)**); here **f** is a NumPy array containing feature information. How many data instances are remaining in the data frame after the data entries containing outliers are eliminated?

Step 5 Put the data into the correct form for the classifiers. Remember that the target array y is just 0 or 1 (acceptable or not). To get the feature data you can either use `vstack` for all 11 features or an easier way is to create a new data frame where you drop the columns that do not contain the feature data and then create a 2D NumPy array from this using, for example, `X = df_mod.values` where `df_mod` is your modified dataframe. Either way, X should be dimensioned by the number of data entries at the end of Step 4 by 11 features.

Step 6 Split the data into training and test sets with an 80-20 split using scikit-learn's function `train_test_learn`. Print out the number of data instances in each set.

Step 7 Scale the data as we have done previously using scikit-learn's `StandardScaler`

Step 8 Apply scikit-learn's logistic regression algorithm to solve this binary classification problem. Be sure to set the solver using `lbfgs`. Print out the percent of accuracy for predicting the test set.

Step 9 Apply scikit-learn's ANN algorithm to solve this binary classification problem. Print out the percent of accuracy for predicting the test set.

Step 10 Apply scikit-learn's kNN algorithm to solve this binary classification problem for values of k=1 to k=5. Determine which choice of k gives the best result. Print out the percent of accuracy for predicting the test set.

Step 11 Compare results from the 3 algorithms.

Extra Credit Apply 1 or more additional algorithms from scikit-learn's library to this binary classification problem and compare with the 3 algorithms in steps 8-10.

```
import numpy as np
import matplotlib.pyplot as plt
from pandas import DataFrame
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.cluster import KMeans
import pandas as pd
import seaborn as sns
df = pd.read_csv('white_wine_quality.csv', sep=';')
```

```
# Step 1 - create dataframe
df['acceptable'] = df['quality'].map(lambda x: 1 if x >= 7 else 0)
```

```
# Step 1 continued.  Is the wine acceptable? 1 = True, 0 = False
print("\nRows where wine SHOULD be acceptable:")
print(df[df["quality"] >= 7][["quality", "acceptable"]].head(10))

print("\nRows where wine SHOULD NOT be acceptable:")
print(df[df["quality"] < 7][["quality", "acceptable"]].head(10))
```

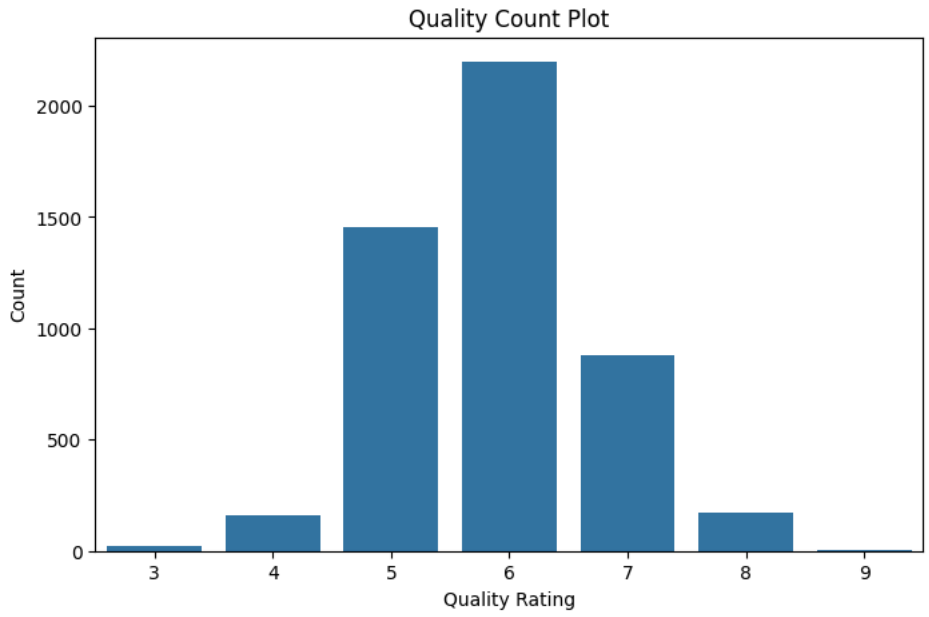
```
Rows where wine SHOULD be acceptable:
    quality  acceptable
13        7           1
15        7           1
17        8           1
20        8           1
21        7           1
22        8           1
29        7           1
45        7           1
51        7           1
52        7           1

Rows where wine SHOULD NOT be acceptable:
   quality  acceptable
0        6           0
1        6           0
2        6           0
3        6           0
4        6           0
5        6           0
6        6           0
7        6           0
8        6           0
9        6           0
```

```
# Step 2. Nan values?
print("\nNumber of NaN values in each column:")
print(df.isna().sum())

#drop rows with NaN values
df = df.dropna()
```

```
Number of NaN values in each column:
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
acceptable              0
dtype: int64
```

```
# Step 3. Quality groupings and countplot
quality_counts = df.groupby("quality").size()

print("Number of wines in each quality class:")
print(quality_counts)

majority_quality = quality_counts.idxmax()
print(f"\nMost common wine quality rating: {majority_quality}")

plt.figure(figsize=(8,5))
sns.countplot(x='quality', data=df)
plt.title('Quality Count Plot')
plt.xlabel("Quality Rating")
plt.ylabel("Count")
plt.show()
```

```
Number of wines in each quality class:
quality
3      20
4     163
5    1457
6    2198
7     880
8     175
9       5
dtype: int64

Most common wine quality rating: 6
```



```
# Step 4. For each feature determine if any data instance is an outlier;
# if it is delete that data instance

 # Example follows for one feature
f = df['fixed acidity'].values

mean = np.mean(f)
std = np.std(f)
count = 0
factor = 5
for i in range(0,len(f)):
    z = ( f[i] - mean ) / std
    if z > factor:
        print(z)
        count = count + 1
        df = df.drop([i])
print("number of data instances dropped is ", count)
```

```
8.705105871848145
5.860769568232091
number of data instances dropped is  2
```

```
# Step 4 for all features
features = [
    'fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar',
    'chlorides', 'free sulfur dioxide', 'total sulfur dioxide',
    'density', 'pH', 'sulphates', 'alcohol'
]

factor = 5

for feature in features:
    f = df[feature].values
    mean = np.mean(f)
    std = np.std(f)
    outlier_indices = []

    for i in range(len(f)):
        z = (f[i] - mean) / std
        if z > factor:
            outlier_indices.append(i)

    df = df.drop(outlier_indices)
    print("number of data instances dropped for feature", feature, "is", len(outlier_indices))
```

```
number of data instances dropped for feature fixed acidity is 0
number of data instances dropped for feature volatile acidity is 9
number of data instances dropped for feature citric acid is 8
number of data instances dropped for feature residual sugar is 1
number of data instances dropped for feature chlorides is 56
number of data instances dropped for feature free sulfur dioxide is 7
number of data instances dropped for feature total sulfur dioxide is 2
number of data instances dropped for feature density is 3
number of data instances dropped for feature pH is 0
number of data instances dropped for feature sulphates is 1
number of data instances dropped for feature alcohol is 0
```

```
# Step 5. get data into correct form
X = df.drop(columns=['quality', 'acceptable']).values
y = df['acceptable'].values
```

```
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (4809, 11)
y shape: (4809,)
```

```
# Step 6. Split data in test and trial sets with 80-20 split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training instances:", X_train.shape[0])
print("Test instances:", X_test.shape[0])
```

```
Training instances: 3847
Test instances: 962
```

```
# Step 7. Scale the data
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
#  Step 8. Logistic Regression
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

logreg = LogisticRegression(solver='lbfgs', max_iter=2000)
logreg.fit(X_train, y_train)
y_pred_log = logreg.predict(X_test)
acc_log = accuracy_score(y_test, y_pred_log)
print("Logistic Regression accuracy:", acc_log*100)
```

```
Logistic Regression accuracy: 80.24948024948026
```

```
# Step 9. Create ANN classifier and train
from sklearn.neural_network import MLPClassifier

ann = MLPClassifier(hidden_layer_sizes=(50,), max_iter=1000, random_state=42)
ann.fit(X_train, y_train)
y_pred_ann = ann.predict(X_test)
acc_ann = accuracy_score(y_test, y_pred_ann)
print("ANN accuracy:", acc_ann*100)
```

```
ANN accuracy: 83.991683991684
```

```
# Step 10. Create kNN classifier and see what value of k is best
from sklearn.neighbors import KNeighborsClassifier

best_k = 0
best_acc = 0

for k in range(1,6):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred_knn = knn.predict(X_test)
    acc_knn = accuracy_score(y_test, y_pred_knn)
    print("k =", k, "kNN accuracy:", acc_knn*100)
    if acc_knn > best_acc:
        best_acc = acc_knn
        best_k = k

print("Best k for kNN:", best_k, "with accuracy:", best_acc*100)
```

```
k = 1 kNN accuracy: 85.03118503118503
k = 2 kNN accuracy: 84.51143451143452
k = 3 kNN accuracy: 82.53638253638253
k = 4 kNN accuracy: 83.05613305613305
k = 5 kNN accuracy: 83.26403326403326
Best k for kNN: 1 with accuracy: 85.03118503118503
```

Compare your results

```
print("\nComparison of algorithms:")
print("Logistic Regression:", acc_log*100)
print("ANN:", acc_ann*100)
print("Best kNN (k={}):".format(best_k), best_acc*100)

print("kNN algorith performed the best with an accuracy of {}%".format(best_acc*100))
print("Logistic Regression performed the worst with an accuracy of {}%".format(acc_log*100))
```

```

Comparison of algorithms:
Logistic Regression: 80.24948024948026
ANN: 83.991683991684
Best kNN (k=1): 85.03118503118503
kNN algorith performed the best with an accuracy of 85.03118503118503%
Logistic Regression performed the worst with an accuracy of 80.24948024948026%
```

```
# Extra credit.
# Apply 1 or more additional algorithms from scikit-learn's library to this binary classification problem
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
acc_rf = accuracy_score(y_test, y_pred_rf)
print("Random Forest accuracy:", acc_rf*100)
```

```
Random Forest accuracy: 88.98128898128898
```